TECHSPECIALIST LIMITED

SYSTEM DOCUMENTATION

AI-Powered Recruitment Platform

A single reference for both audiences: a plain-language business overview for executives and stakeholders, followed by a complete technical reference for engineers and technical due diligence.

Prepared for internal and stakeholder distribution

Business overview and full technical reference

July 2026

Contents

PART ONE · EXECUTIVE SUMMARY

A Recruitment Platform Built Around AI, Not Bolted To It

TechSpecialist Limited has built and now operates a complete, AI-powered recruitment platform. It is not a plug-in added to an existing hiring process. It was designed from the ground up so that artificial intelligence does the early, repetitive work of hiring: reading every application, holding a live spoken interview with qualified candidates, and producing a structured, evidence-based recommendation, so that HR's time goes to judgment and decisions, not administration.

What this document is

This document is written for two audiences. Part One, which you are reading now, explains what the platform does and why it matters, in plain business language, for executives, board members, and other stakeholders who need to understand the system without reading code. Part Two is a complete technical reference, covering architecture, data, configuration, security, and operations, for engineers, technical partners, or anyone conducting technical due diligence.

Read Part One for the business picture. Read all of it for the full technical detail.

> **In one sentence:** every job application is read and scored by AI within moments of being submitted, strong candidates are interviewed live by an AI interviewer that holds a natural spoken conversation, and HR reviews a structured recommendation, not raw data, before making every final decision.

PART ONE · THE BUSINESS CASE

Why This Exists

The problem with traditional hiring

Reviewing every CV against a role's actual requirements takes real time, and that time does not scale as the number of applications grows. A first-round phone or video screen, run by a person, takes thirty minutes to an hour per candidate, multiplied by every candidate who applies. In most organizations, that leads to one of two outcomes: hiring teams fall behind, or they cut corners on how carefully each candidate is actually considered.

What the platform changes

Every application submitted through the careers page is read by AI within moments, scored against the specific requirements of that role, and handed to HR with a written rationale, not just a number. Candidates who clear that first stage move to a short, live, AI-conducted voice interview, held automatically or on HR's approval, that asks role-relevant questions and adapts its follow-up questions to what the candidate actually says. HR then reviews a structured recommendation and supporting evidence, not a raw transcript, before making the final call.

The AI never makes a hiring decision. It produces evidence and a recommendation. Every action taken on a candidate, whether initiated by a person or triggered automatically by a configured rule, is permanently recorded.

PART ONE · HOW IT WORKS

The Hiring Pipeline, Plainly Explained

From a candidate's first click to a final hiring decision, here is what actually happens:

1. **Job Posted**
   HR creates a listing, including how the AI should screen applicants for it and what it should cover in the interview.

2. **Candidate Applies**
   CV and cover letter, submitted directly on the website. No account required.

3. **AI Screens the CV**
   Every application is scored against that role's specific requirements within moments, with a written explanation of strengths and concerns, not just a number.

4. **HR Reviews and Advances**
   HR approves strong candidates to the next stage, or configures the role so qualifying candidates advance automatically, on HR's own terms.

5. **Live AI Interview**
   Qualified candidates have a short, natural, spoken conversation with the AI interviewer, covering topics relevant to the role.

6. **AI Produces a Recommendation**
   A structured scorecard and a written recommendation, ready for a human to review, not a decision already made.

7. **HR Decides**
   A final human interview, structured feedback from every interviewer involved, and the hiring decision itself.

> At every stage, what happened and why is written to a permanent record, so the process stays fair, consistent, and defensible.

PART ONE · PLATFORM CAPABILITIES

What Has Been Built

A working system, in production use today, with the following capabilities:

AI CV Screening

Every application evaluated consistently against a role's actual requirements, with written strengths and concerns, not a black-box score.

Live AI Voice Interview

A real, spoken conversation, not a scripted quiz. The AI listens, responds naturally, and adapts its questions as the conversation develops.

Automatic Advancement Rules

Roles can be configured so strong candidates move to interview automatically, on HR's terms, with a configurable pass mark and delay.

Visual Pipeline Board

HR sees every candidate's stage at a glance and moves candidates through the process directly, not through spreadsheets.

Structured Interviewer Scorecards

Multiple interviewers can score the same candidate against the same criteria, reducing gut-feel bias in the final decision.

Calendar-Integrated Scheduling

Final interviews are scheduled with an automatic calendar invitation, with no manual back-and-forth needed.

Analytics and Reporting

Pipeline health, time-to-hire, and interview outcomes, visible to leadership at any time.

Full Audit Trail

Every decision made on every candidate is permanently recorded, supporting fair and defensible hiring practices.

Candidate Self-Service

Candidates receive instant confirmation and can check their own status at any time, without calling or emailing HR.

Team Management

HR can invite and manage its own team's accounts directly, with sensible safeguards against locking the team out.

PART ONE · COMPETITIVE POSITION

How This Compares to Commercial Hiring Software

Most established recruiting platforms, the well-known names that dominate the market, were built years before generative AI existed. Many are now adding AI features, but as an add-on layered onto a workflow originally designed around forms and manual review. This platform was built the other way around: AI screening and a live AI voice interview are native to how the system works, not a bolted-on integration.

That is a genuine and current advantage. It is equally honest to say what a platform like this does not yet have, because the established players have spent years building it: support for many separate client organizations on one platform, enterprise login standards that large companies require, formal compliance certifications, and integrations with external job boards and background-check providers.

> None of that is needed for how the platform is used today, inside TechSpecialist Limited. All of it becomes relevant only if and when the decision is made to offer this platform to other companies. See "If This Becomes a Product," below.

PART ONE · RECENT INVESTMENT

Recently Completed Work

The platform is under active, ongoing development. The most recent round of work focused on two things: giving HR more control over the pipeline, and making the AI interview experience more reliable and natural for candidates.

HR PIPELINE IMPROVEMENTS

- A visual, drag-and-drop pipeline board, replacing manual status tracking
- Bulk actions to archive, reject, or reactivate multiple candidates at once
- Automatic flags for applications that have stalled or appear to be duplicates
- Structured, multi-interviewer scorecards
- Calendar invitations attached automatically to interview scheduling emails
- A candidate self-service status page
- Configurable automatic-advancement rules, with a pass mark and delay HR controls per role

AI INTERVIEW RELIABILITY

- Following direct feedback from a live HR demonstration, the interview engine was rebuilt to correctly handle background noise and natural conversational pauses, so candidates are no longer cut off mid-thought
- The interview now closes gracefully, only after the AI has finished speaking, instead of ending abruptly
- On-screen captions now appear in sync with what the AI is saying, in real time
- Each role now has a configurable maximum interview length, with a visible countdown for the candidate and a graceful wrap-up instead of a silent cutoff

PART ONE · LOOKING AHEAD

If This Becomes a Product

Leadership has expressed interest in eventually offering this platform commercially, beyond TechSpecialist Limited's own hiring. The platform's AI-native core is a genuine head start toward that goal. What stands between today and a commercial launch is not the AI. It is the enterprise scaffolding that any multi-customer product needs:

- **Multi-tenancy** — supporting many separate companies securely on one platform, instead of the single-company setup that exists today.
- **Enterprise single sign-on** — the login standard large customers require before they will adopt new software.
- **Formal compliance certification** — for data protection and equal-opportunity hiring regulations, verified by an independent auditor.
- **External integrations** — job boards, background-check providers, and existing HR systems that a commercial customer would expect to connect.

None of this needs to happen before it is needed. It is listed here so the path forward, and the scope of work it represents, is visible to whoever is making that decision.

> **What follows** is a complete technical reference: system architecture, data storage, every configuration variable and third-party integration, feature-by-feature technical walkthroughs, the full API surface, security posture, and deployment operations. It is written for engineers, technical partners, or anyone conducting technical due diligence, and it is kept current against the actual code, not the original design intent.

PART TWO

Technical Reference

The remainder of this document mirrors the engineering team's living documentation, covering architecture, data model, configuration, feature-by-feature technical walkthroughs, the API surface, security posture, and deployment operations. It describes what the code actually does today, including verified gaps, not what it was originally intended to do.

TECHNICAL FRAMING

Executive Overview

What this system is

TechSpecialist Limited's web presence is not just a marketing website — it's a working business application with three distinct products living under one domain:

1. **The public website** — homepage, services, case studies, insights (blog), consultation booking, legal pages, and a privacy/data-deletion page for a separate mobile app ("ICE", built for NSIA).
2. **An AI-powered recruitment platform** — a careers board where candidates apply for jobs, an AI that automatically screens every CV against the job requirements, an AI that conducts a first-round voice interview with shortlisted candidates, and an internal HR portal where staff manage jobs, review candidates, schedule final interviews, and see recruitment analytics.
3. **An AI Readiness Assessment** — a public, self-serve quiz that scores a visitor's organization on "AI readiness," used as a marketing lead-generation tool, plus an internal dashboard to review the leads it captures.

Who uses it, and how

USER	WHAT THEY DO	WHERE
Website visitor / prospect	Browses services and case studies, books a discovery call, takes the AI Readiness quiz	Public pages
Job candidate	Finds a job, applies with CV + cover letter, later completes an AI voice interview by email link	`/careers`, `/apply`, `/assessment/[token]`
HR staff	Posts jobs, reviews AI-screened candidates, approves/rejects, schedules final interviews, views analytics, manages other HR accounts	`/hr/*` (login required)
Marketing/sales staff	Reviews AI Readiness quiz leads, exports them, marks them as followed up	`/admin/assessments`

The recruitment pipeline, in plain terms

This is the most complex part of the system, and the one that does the most work:

```
1. HR posts a job (including whether stage 2 is manual-approve or auto-advance)
        ↓
2. Candidate applies (CV + cover letter, no account needed)
        ↓
3. AI reads the CV against the job requirements and produces a score + written strengths/concerns
        ↓
4. HR reviews the AI's assessment and approves or rejects — or, if the job is configured for
   auto-advance and the score clears the pass mark, this step happens automatically after a delay
        ↓
5. The candidate gets an emailed link to a short AI-conducted live voice interview
        ↓
6. The AI interviews the candidate by voice in real time, topic by topic, and produces a scorecard
   plus a written recommendation, surfaced to HR as a "pending decision" needing a human call
        ↓
7. HR reviews the scorecard, schedules a final human interview (with a calendar invite),
   collects one or more interviewer scorecards, and marks the candidate hired or rejected
```

The AI never makes a final hiring decision — it only produces scores and recommendations that a human reviews at checkpoints throughout (steps 4, 6, and 7). Every action taken on a candidate (approved, rejected, hired, interview scheduled, archived, deleted, flagged for suspicious behavior during the AI interview) is written to an audit log. Candidates can also check their own application status at any time via a self-service link, without contacting HR.

Major components (non-technical summary)

- **The website you see** is built with Next.js, a modern web framework, and is what visitors' and candidates' browsers load directly.
- **A separate backend service** (built in Python) does the actual work of storing recruitment data, running the AI screening, and running the AI voice interviews. The website talks to this backend behind the scenes.
- **Two databases exist**: the Python backend's own database (the primary store for jobs, applications, interviews, HR accounts), and a Supabase (cloud Postgres) database that the website falls back to for job listings and applications if the backend is temporarily unreachable. See 02-data-model-and-storage.md for why there are two, and what that implies.

- **AI providers**: Azure OpenAI (GPT-4o) powers CV screening; Azure OpenAI's Realtime API (`gpt-realtime`) powers the live, voice-to-voice AI interview — the candidate speaks and hears the AI respond in real time, the same way a phone screen would work, rather than a record-then-reply exchange; Anthropic's Claude (with Groq as a fallback) generates the AI Readiness Assessment reports. These are different AI systems for different features.
- **Email** is sent through Resend (recruitment notifications, reports) and EmailJS (marketing lead notifications on the public site).

What to know before relying on this system

A full, itemized list of verified gaps is in 06-security-and-known-gaps.md, but the two most important for decision-makers:

- **The AI Readiness Assessment leads dashboard (`/admin/assessments`) has no login or access control at all.** Anyone who finds the URL can view, export, or delete the captured leads.
- **Some recruitment data-entry paths bypass authentication** when the Python backend is unreachable, because the website's fallback path writes straight to Supabase without checking who's asking.

Neither of these affects candidates' CVs being screened incorrectly or interviews being scored unfairly — they are access-control gaps in administrative surfaces, not flaws in the AI logic itself.

ARCHITECTURE

Architecture

Summary

This is a **two-application system**, not one. A Next.js frontend serves everything a browser talks to; a separate Python (FastAPI) backend owns the recruitment database and runs the AI logic. The frontend calls the backend over HTTP and WebSocket. For two specific recruitment flows, the frontend also has a direct fallback to Supabase, so it can keep working (in a degraded way) if the Python backend is down.

Why two applications instead of one

The Next.js app could theoretically do everything (Next.js supports API routes). Instead, all the AI-heavy, stateful recruitment logic — CV screening, the voice-interview engine, HR auth, analytics — lives in a separate Python backend with its own database. The Next.js `api/recruitment/**` routes are, almost without exception, **thin proxies**: they forward the request to the Python backend and return its response. The one shared helper for this is `src/app/api/recruitment/proxy.ts` (`proxyToBackend()`).

This split matters for anyone maintaining the system:

- **Recruitment business logic changes happen in `backend/`, not `src/`.** If you need to change how CV screening scores, how the interview state machine works, or how HR permissions are enforced, look in `backend/app/services/` and `backend/app/routers/`, not the Next.js API routes.
- **The Next.js layer's main jobs are:** rendering UI, proxying requests, and providing a Supabase-backed fallback for two specific write paths (job listing, application submission) so the public careers page degrades gracefully instead of hard-failing if the backend is briefly down.
- **The AI Readiness Assessment is entirely separate** from the recruitment platform's AI — it lives in the Next.js layer (`src/lib/ai-report.ts`, calling Anthropic/Groq directly) and in one backend router (`ai_readiness.py`) purely for storing lead submissions. It does not touch the recruitment database's core tables.

Request flow examples

A visitor loads the careers page: `GET /careers` → `src/app/careers/page.tsx` → `fetchJobs()` → `GET /api/recruitment/jobs` → `proxyToBackend()` → backend `GET /api/jobs`. If that fails, the Next.js route handler falls back to querying Supabase's `jobs` table directly.

A candidate submits an application: Browser → `POST /api/recruitment/applications` → proxied to backend `POST /api/applications` (protected by a shared `x-api-key` secret, not a user login) → backend extracts CV text, uploads files to storage, runs AI screening **synchronously in the same request** (see 07-deployment-and-operations.md for why this is worth knowing), writes the `Application` + `AIScreeningResult` rows, sends notification emails. If the backend call fails, the Next.js layer falls back to writing directly into Supabase's `applications` table and storage bucket instead — meaning that application would **never get AI-screened** (see 06-security-and-known-gaps.md).

A candidate takes the AI voice interview: Browser → `wss://.../api/assessment/{token}/realtime-ws` → backend holds one persistent `gpt-realtime` session open for the whole interview (voice-to-voice, semantic VAD turn detection) → on completion, backend scores the conversation and writes a `StageResult` row. The legacy HTTP/WebSocket (Whisper/GPT-4o/TTS) engine still exists and is reachable at `/api/assessment/{token}/ws`, but is not what candidates get today — see 04-features/03-ai-voice-interview.md.

Environments

ENVIRONMENT	FRONTEND	BACKEND	BACKEND DB
Local dev	`npm run dev0` (`next dev`)	`uvicorn app.main:app --reload`, `RECRUITMENT_API_URL=http://localhost:8000`	SQLite (`backend/recruitment.db`), `dev_mode` true, local-disk file storage under `backend/storage/`
Production	Azure Web App (Node/Next.js), per CI workflow	Azure Web App for Containers `techspecialist-api` (resource group `techspecialist`), image `techspecialistacr.azurecr.io/recruitment-api:latest`	Postgres (per `backend/.env.example`), Azure Blob Storage

Gap worth flagging: there is no CI/CD workflow in `.github/workflows/` for the `backend/` service, and no `docker-compose.yml` tying frontend and backend together. The backend has a `Dockerfile` (`python:3.12-slim`, exposes port 8000), and deployment is a **manual, human-run process**: `az acr build --registry techspecialistacr --image recruitment-api:latest ./backend` (rebuilds and pushes the image from the local `backend/` directory — not from git, so uncommitted local changes get deployed too if you're not careful), then `az webapp restart --name techspecialist-api --resource-group techspecialist`. See 07-deployment-and-operations.md for the full sequence and a real operational quirk (the first restart after a build sometimes still serves the old cached image — verify a field/behavior only the new code has before declaring the deploy done, don't just trust `/health`).

A stray `out/` directory exists at the repo root (looks like a static export from `next export` / `output: 'export'` at some point), but the current `next.config.mjs` has no `output: 'export'` setting and `package.json`'s `build` script is plain `next build`. Treat `out/` as a stale build artifact unless someone confirms otherwise — it's not part of the live build pipeline.

DATA MODEL & STORAGE

Data Model and Storage

Summary

There are **two separate databases** and **two separate file-storage systems**, and they don't talk to each other:

1. **The Python backend's own database** — the primary, authoritative store for all recruitment data (jobs, applications, screening results, interviews, HR accounts). Postgres in production, SQLite locally.
2. **Supabase (Postgres)** — used by the Next.js frontend only as a fallback, holding a `jobs` table and an `applications` table that can drift out of sync with the backend's real tables.

This split exists because the frontend was built to degrade gracefully if the backend is briefly unreachable, but it means **an application submitted during a backend outage lands in Supabase, not the backend database, and never gets AI-screened or shows up in the HR portal.** This is the single most important storage fact to know about this system — see the callout at the bottom of this document.

1. Primary database (Python backend)

SQLAlchemy 2.0 async ORM. Dev default `sqlite+aiosqlite:///./recruitment.db` (file: `backend/recruitment.db`); production uses `postgresql+asyncpg://...` via the `DATABASE_URL` env var. Tables are created from the ORM models on startup (`Base.metadata.create_all`); schema patches since initial deployment are applied by hand-rolled `ALTER TABLE` statements in `backend/app/migrate.py` (there is **no Alembic migration history** despite the `backend/alembic/` directory existing — it's empty; see 07-deployment-and-operations.md).

All tables use a UUID primary key unless noted.

`job_postings` (model: `JobPosting`, file: `backend/app/models/job_posting.py`)

A job listing HR has created.

FIELD	TYPE	NOTES
`id`	UUID PK	
`title`	string	
`description`	text	
`requirements`	text	
`department`	string(120)	
`location`	string(120)	
`type`	string(50)	default "Full-time"
`screening_instructions`	text	tells the AI what to weigh when scoring CVs for this role
`stage2_instructions`	text	system instructions for the AI voice interviewer
`stage2_questions`	text	JSON-encoded list of seed questions
`stage2_topic_labels`	text	JSON-encoded list of topic labels for the interview
`status`	string(20)	default "active"
`is_deleted`	bool	soft-delete flag
`is_closed`	bool	closed to new applications, but not deleted
`auto_advance_enabled`	bool	default false — when true, candidates who clear `auto_advance_pass_mark` on CV screening skip the manual HR approval step for stage 2
`auto_advance_pass_mark`	float	default 70.0 — CV screening score threshold for auto-advance
`auto_advance_delay_minutes`	int	default 5 — delay before the stage-2 invite email actually sends after auto-advance triggers (gives HR a window to intervene manually)
`interview_max_minutes`	int	default 20 — soft target for the AI voice interview's length; the model is nudged to wrap up gracefully as this approaches (see 04-features/03-ai-voice-interview.md). A separate backend-wide hard cap (`realtime_max_session_seconds`, default 2700s) always applies underneath regardless of this value.

FIELD	TYPE	NOTES
`created_at` / `updated_at`	timestamp	

Relationship: one `JobPosting` → many `applications`.

`applications` (model: `Application`, file: `application.py`)

One candidate's application to one job. The central record of the recruitment pipeline.

FIELD	TYPE	NOTES
`id`	UUID PK	
`job_id`	UUID FK → `job_postings.id`	
`candidate_name`, `candidate_email`	string(255)	
`cv_url`, `cover_letter_url`	text	storage URL (see storage section below)
`cv_text`, `cover_letter_text`	text, nullable	extracted plain text used for AI screening
`status`	string(20)	`pending`, `approved`, `rejected`, `hired`, `assessment_completed`, `assessment_flagged`, `interview_scheduled`
`stage`	int	default 1 (1 = CV screen, 2 = AI voice interview, 3 = human interview)
`is_archived`	bool	default false — set via bulk archive/unarchive in the HR portal; archived applicants are hidden from the default list/board view but not deleted
`assessment_token`	string(255), unique, nullable	magic-link token emailed to the candidate for stage 2
`assessment_sent_at`, `assessment_expires_at`	timestamp, nullable	
`created_at` / `updated_at`	timestamp	

Relationships: belongs to one `JobPosting`; has one `AIScreeningResult`; has many `StageResult`; has one `ConversationSession`; has many `Interview`; has many `InterviewScorecard` (via backref); has many `AuditLog` (not a formal FK).

Note: the candidate email can be looked up across applications to flag likely duplicates in the HR portal (`application_count_for_email` computed field) — see 04-features/04-hr-portal.md.

`ai_screening_results` (model: `AIScreeningResult`, file: `ai_result.py`)

The AI's stage-1 verdict on a CV, one per application.

FIELD	TYPE	NOTES
`id`	UUID PK	
`application_id`	UUID FK → `applications.id`	
`overall_score`	float	
`strengths`, `concerns`, `evidence`	text, nullable	
`raw_response`	text, nullable	the AI's raw output, kept for auditability
`created_at`	timestamp	

`stage_results` (model: `StageResult`, file: `stage.py`)

The outcome of a completed stage-2 AI voice interview session.

FIELD	TYPE	NOTES
`id`	UUID PK	
`application_id`	UUID FK	
`stage_number`	int	
`status`	string(20)	default "pending"
`audio_url`	text, nullable	
`transcript`	text, nullable	JSON-encoded conversation

FIELD	TYPE	NOTES
`score`	float, nullable	
`ai_feedback`	text, nullable	JSON-encoded multi-dimension evaluation (communication, technical competency, confidence, problem-solving, relevance)
`created_at` / `updated_at`	timestamp	

`conversation_sessions` (model: `ConversationSession`, file: `conversation.py`)

The **live, in-progress** state of an AI voice interview — not the final result (that's `stage_results`).

FIELD	TYPE	NOTES
`id`	UUID PK	
`application_id`	UUID FK	
`status`	text	default "in_progress"; also `completed`, `terminated_violation`
`conversation_history`	text	JSON list of `{role, content, topic_label}`
`current_topic_index`, `turns_on_current_topic`	int	interview state-machine bookkeeping
`topics`	text	JSON list of `{label, seed_question}`, copied from the job's `stage2_topic_labels` / `stage2_questions` at session start
`engine`	text	`"legacy"` (HTTP/WebSocket, Whisper+GPT-4o+TTS pipeline) or `"realtime"` (Azure OpenAI Realtime API, voice-to-voice)
`created_at` / `updated_at`	timestamp	

`interviews` (model: `Interview`, file: `interview.py`)

A **human-scheduled** final interview (stage 3) — distinct from the AI voice interview.

FIELD	TYPE	NOTES
`id`	UUID PK	
`application_id`	UUID FK	
`interview_type`	string(20)	`physical` or `virtual`
`scheduled_date`, `scheduled_time`	string	
`duration_minutes`	int	default 60
`location`, `meeting_link`, `interviewer_name`, `notes`	nullable	
`status`	string(20)	default "scheduled"
`created_at` / `updated_at`	timestamp	

`interview_scorecards` (model: `InterviewScorecard`, file: `scorecard.py`)

A structured scoring form an interviewer fills out after the stage-3 human interview. Multiple scorecards can exist per application (one per interviewer).

FIELD	TYPE	NOTES
`id`	UUID PK	
`application_id`	UUID FK → `applications.id`	
`interviewer_name`	string(255)	
`communication_score`, `technical_score`, `culture_fit_score`, `problem_solving_score`	int	1–5 scale each, entered via sliders in `ScorecardPanel.tsx`
`recommendation`	string(20)	e.g. "strong_yes", "yes", "no", "strong_no"
`notes`	text, nullable	
`created_at`	timestamp	

`audit_logs` (model: `AuditLog`, file: `audit_log.py`)

Compliance/history trail — every meaningful action taken on an application.

FIELD	TYPE	NOTES
`id`	UUID PK	
`application_id`	UUID, indexed (not a formal FK)	
`action`	string(50)	e.g. `assessment_approved`, `rejected`, `interview_scheduled`, `interview_terminated_violation`, `application_cleared`
`detail`	text, nullable	
`performed_by`	string(255), nullable	
`created_at`	timestamp	

`hr_users` (model: `HrUser`, file: `hr_user.py`)

HR staff accounts — this backend has its own login system, entirely separate from Supabase.

FIELD	TYPE	NOTES
`id`	UUID PK	
`email`	string(255), unique	
`name`	string(255)	
`password_hash`	string(255)	bcrypt
`is_active`	bool	default true
`reset_token_hash`, `reset_token_expires_at`	nullable	forgot-password / invite-a-new-user flow
`created_at` / `updated_at`	timestamp	

`app_settings` (model: `AppSetting`, file: `app_setting.py`)

Runtime-editable key/value store for HR-configurable branding, overlaid onto the app's in-memory settings at startup and whenever HR edits `/hr/settings`.

FIELD	TYPE	NOTES
`key`	string(100) PK	`company_name`, `brand_color`, `logo_url`, `sender_display_name`, `hr_notification_email`
`value`	text	
`updated_at`	timestamp	

`ai_readiness_assessments` (model: `AiReadinessAssessment`, file: `ai_readiness_assessment.py`)

Lead-magnet quiz submissions. **Unrelated to the recruitment tables above** — no foreign keys connect them.

FIELD	TYPE	NOTES
`id`	UUID PK	
`email`	string(255), indexed	
`company_name`	string(255)	default "Not provided"
`scores`	text	JSON dict, per-pillar scores
`total_score`	float	
`max_score`	float	default 75
`level`	string(50)	e.g. "AI Explorer", "AI Builder", "AI Accelerator", "AI Leader"
`followed_up`	bool	default false — toggled from `/admin/assessments`
`created_at`	timestamp	

2. Supabase (fallback database, used by the Next.js frontend)

Client: `src/lib/supabase.ts`, using the **anon key** (`NEXT_PUBLIC_SUPABASE_ANON_KEY`) — no service-role key is used anywhere in the frontend. Only two tables are touched from `src/`:

- **`jobs`** — read by `/careers`, `/careers/[id]`, `/apply`, and `GET /api/recruitment/jobs`, as a fallback when the backend's `GET /api/jobs` fails. Also written by the HR jobs API fallback (`src/app/api/recruitment/hr/jobs/route.ts`) when the backend is unreachable.

- `applications` — written by the legacy `POST /api/apply` route and by `POST /api/recruitment/applications`'s fallback path, when the backend is unreachable.

These are not guaranteed to have the same schema as the backend's `job_postings` / `applications` tables — they were built independently as a resilience fallback, not as a replica. Nothing in this repository keeps them in sync. Treat any data that ends up here (rather than in the backend's own database) as a case that needs manual reconciliation.

Supabase Storage

One bucket, `applications`, holding:

- `resumes/<uuid>.<ext>` — candidate CVs uploaded via the fallback path
- `cover-letters/<uuid>.<ext>` — candidate cover letters uploaded via the fallback path

3. Backend file storage (CVs, interview audio)

Independent of Supabase Storage. Controlled by `backend/app/services/storage.py`:

- **Local dev / no Azure Storage configured:** files are written to `backend/storage/<container>/` on local disk, referenced in the database as a `file://<path>` URL. Confirmed present: `backend/storage/cvs/` (candidate CV/cover-letter text extracts, UUID-named `.txt` files).
- **Production (Azure Storage connection string configured):** files go to Azure Blob Storage, in containers named by `cvs_container_name` (default `cvs`) and `assessments_container_name` (for interview-answer audio).

The two-database problem — read this before changing anything

Because the Next.js frontend has an independent Supabase fallback for `jobs` and `applications`, **the backend's Postgres/SQLite database is not guaranteed to be the single source of truth** for those two entities. Concretely:

- A job created via `/hr/jobs/create` while the backend is reachable goes into the backend's `job_postings` table (correct, full-featured).
- A job listed on `/careers` when the backend happens to be down for that one request is read from Supabase's `jobs` table instead — which may be stale or missing rows the backend has.
- An application submitted while the backend is down goes into Supabase's `applications` table, **skips AI screening entirely**, and will not appear anywhere in the HR portal (which only reads from the backend database) until someone notices and manually moves it over.

If you're troubleshooting "a candidate says they applied but HR can't see them," check Supabase's `applications` table, not just the backend database. This is documented further in 06-security-and-known-gaps.md.

CONFIGURATION & INTEGRATIONS

Configuration and Integrations

Summary

The system depends on roughly a dozen third-party services split across the frontend and backend: Supabase, Azure OpenAI (four different capabilities), Azure Blob Storage, Resend, Anthropic, Groq, and EmailJS. Every credential is environment-variable-driven except EmailJS's, which are hardcoded in source. This document lists every variable, what it's for, and where it's read — **values are intentionally not reproduced here**; find them in each environment's actual `.env` file or secrets manager.

Frontend environment variables (`.env.local`, or platform secrets in production)

VARIABLE	PURPOSE	READ IN
`NEXT_PUBLIC_SUPABASE_URL`	Supabase project URL	`src/lib/supabase.ts`
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase anon (public) key — no service-role key is used	`src/lib/supabase.ts`
`RECRUITMENT_API_URL`	Base URL of the Python backend (default `http://localhost:8000`)	`src/app/api/recruitment/proxy.ts` and all recruitment API routes
`RECRUITMENT_API_KEY` / `API_KEY`	Shared secret sent as `x-api-key` when proxying the public application-submit call	`src/app/api/recruitment/applications/route.ts`
`HR_EMAIL`, `HR_PASSWORD`	**Fallback** HR login credentials used only if the backend is unreachable during login (see 06-security-and-known-gaps.md)	`src/app/api/recruitment/hr/auth/route.ts`
`JWT_SECRET`	Used only by the fallback token-minting path above — not real HMAC signing, string concatenation only	same file
`GROQ_API_KEY`	Fallback AI provider for AI Readiness report generation	`src/lib/ai-report.ts`
`ANTHROPIC_API_KEY`	Primary AI provider for AI Readiness report generation (Claude)	`src/lib/ai-report.ts`
`RESEND_API_KEY`	Sends the AI Readiness PDF report email	`src/app/api/assessment/send-report/route.ts`
`NEXT_PUBLIC_ASSESSMENT_WS_URL`	WebSocket URL for the AI voice-interview portals (default `ws://localhost:8000`)	`RealtimeAssessmentPortal.tsx`, `LegacyAssessmentPortal.tsx`

Present in `.env.local` but **not referenced anywhere in `src/`** — confirmed dead/unused:

- `ADMIN_SECRET_TOKEN` — appears to be a vestige of a planned-but-never-implemented `/admin` auth check. If you're asked "is `/admin/assessments` protected by this token," the answer is no — see 06-security-and-known-gaps.md.
- `VERCEL_OIDC_TOKEN` — Vercel-platform-injected, unrelated to app code.

Not environment-driven: `src/lib/emailjs.ts` hardcodes its public key, service ID, and template ID as string literals rather than reading them from env vars. This is lower-risk than it sounds (EmailJS's "public key" is designed to be exposed client-side), but it does mean rotating these values requires a code change, not a config change.

Backend environment variables (`backend/.env`, documented with placeholders in `backend/.env.example`)

Defined in `backend/app/config.py` (Pydantic settings, loaded from `.env`).

GROUP	VARIABLES	PURPOSE
App	`app_name`, `cors_origins`, `dev_mode`	`dev_mode` toggles local-disk storage, console-logged emails, and silent TTS instead of hitting real Azure endpoints
Database	`database_url`	Postgres in prod, SQLite locally
Azure OpenAI — chat	`azure_openai_endpoint`, `azure_openai_key`, `azure_openai_api_version`, `gpt4o_deployment_name`	Powers CV screening and the "legacy" interview engine's conversation logic
Azure OpenAI — Whisper	`azure_whisper_endpoint`, `azure_whisper_key`, `azure_whisper_api_version`, `whisper_deployment_name`	Speech-to-text for the legacy voice interview engine

GROUP	VARIABLES	PURPOSE
Azure OpenAI — TTS	`azure_tts_endpoint`, `azure_tts_key`, `azure_tts_api_version`, `tts_deployment_name`, `tts_voice`	Text-to-speech for the legacy voice interview engine
Azure OpenAI — Realtime	`azure_realtime_endpoint`, `azure_realtime_key`, `realtime_deployment_name`, `realtime_voice`, `realtime_vad_eagerness`, `realtime_noise_reduction_type`, `realtime_max_session_seconds`, `realtime_interview_enabled`	Voice-to-voice interview engine — this is the **primary** interview experience in production (`realtime_interview_enabled=True`), not an experimental flag. `realtime_vad_eagerness` (`low` / `medium` / `high` / `auto`) controls semantic turn detection (how long the model waits before assuming the candidate is done speaking); `realtime_noise_reduction_type` (`near_field` / `far_field`) filters ambient audio before it reaches VAD. `realtime_max_session_seconds` is a hard backend-wide safety cap (default 2700s) independent of the per-job `interview_max_minutes` soft target — see 04-features/03-ai-voice-interview.md
Interview tuning	`topic_time_limit_seconds`, `max_turns_per_topic`	Shared by both interview engines' state machines
Redis	`redis_url`	**Declared but unused** — see 07-deployment-and-operations.md; no Celery task or worker process actually exists
Storage	`azure_storage_connection_string`, `cvs_container_name`, `assessments_container_name`	Azure Blob Storage; falls back to local disk if unset
Branding	`company_name`, `brand_color`, `logo_url`	Defaults, overridden at runtime by the `app_settings` DB table
Email	`resend_api_key`, `sender_email`, `sender_display_name`, `hr_notification_email`, `applicant_reply_to_email`	All recruitment transactional email. `sender_email` is a real repliable address (`recruitment@techspecialistlimited.com`, not a "do not reply" address). `applicant_reply_to_email` (comma-separated) sets the `Reply-To` header on all 7 applicant-facing emails (application received, stage-2 invite, rejection, final-interview invite, hired, expired-link notice) so replies land with a real person rather than nowhere — currently `Taofeeq@mswitchgroup.com,HR@mswitchgroup.com`. The 3 purely-internal emails (new-applicant alert, HR portal invite, password reset) are intentionally left non-repliable (`_branded_template(..., repliable=False)`) since they already go to real monitored inboxes
Auth	`api_key`, `jwt_secret`, `hr_password`	`api_key` gates the public application-submit endpoint; `jwt_secret` signs real HR login JWTs (HS256); `hr_password` seeds the default HR account on first run
Frontend	`frontend_url`	Used to build links in outgoing emails (e.g. "review this candidate" links)

Third-party services — what each one is for

SERVICE	USED BY	PURPOSE
Supabase	Frontend	Fallback datastore for `jobs` / `applications` (see 02-data-model-and-storage.md); anon key only, no auth usage
Azure OpenAI (GPT-4o)	Backend	CV screening scoring, legacy interview conversation logic
Azure OpenAI (Whisper)	Backend	Speech-to-text for the legacy voice interview
Azure OpenAI (TTS)	Backend	Text-to-speech for the legacy voice interview
Azure OpenAI (Realtime API)	Backend	Voice-to-voice AI interview engine — the primary interview experience in production, not just the newer of two options
Azure Blob Storage	Backend	Production file storage for CVs and interview audio
Resend	Both	Backend: all recruitment transactional email (approvals, rejections, invites, password resets). Frontend: AI Readiness Assessment PDF report delivery
Anthropic (Claude)	Frontend	Primary generator of AI Readiness Assessment report content (model: `claude-sonnet-4-6`, structured JSON output)
Groq	Frontend	Fallback generator for the same report if Anthropic isn't configured (model: `llama-3.3-70b-versatile`)
EmailJS	Frontend	Client-side-only marketing lead notifications (discovery call requests, AI Readiness quiz leads) — does not touch the backend at all

Two separate AI systems are in play and are easy to conflate: **Azure OpenAI** does all recruitment AI work (screening, interviews); **Anthropic/Groq** only generate the AI Readiness Assessment's marketing report. They share no code path.

Auth secrets — where they live and what they protect

See 06-security-and-known-gaps.md for the full picture, but as a configuration reference:

- Backend HR JWTs are signed with `jwt_secret` (backend env) — this is the **real** auth mechanism.

- The frontend's `JWT_SECRET` / `HR_EMAIL` / `HR_PASSWORD` are a **separate, weaker fallback** that only activates if the frontend can't reach the backend during login. Keep these two sets of credentials in sync deliberately if you rotate one — they are not the same secret and rotating only one will not fully lock out the fallback path.

Local `.env` gotcha worth knowing

The repo's local `backend/.env` has `dev_mode=False` with a real `resend_api_key` configured — this means running the backend locally sends **real emails through Resend**, not simulated/console-logged ones (that only happens when `dev_mode=True`, or no key is set). Anyone testing recruitment flows locally (applications, approvals, interview scheduling) should be aware every email actually sends. Safe targets for local testing: addresses on IANA-reserved test domains like `@example.com` (accepted by Resend's API but never deliver anywhere), or a real inbox you intend to check.

FEATURE — CAREERS & APPLICATION

Careers Board and Application Form

What it does

Lets any visitor browse open roles and apply with a CV and optional cover letter — no account required.

Walkthrough

1. `/careers` (`src/app/careers/page.tsx`) loads the job list via `fetchJobs()` (`src/lib/recruitment-api.ts`), which calls `GET /api/recruitment/jobs` . That route proxies to the backend's `GET /api/jobs` ; if the backend is unreachable, it falls back to reading Supabase's `jobs` table directly. Only jobs with `status === 'active' && !is_closed` are shown. Job cards show department, location, and job type alongside the title, without the page reading as overcrowded. The page structure is: a slim title bar (not a large hero) → job listings first, above the fold → a "Why Join Us" section (relocated mission copy/stats) below the listings — this ordering was deliberately changed so job openings are the first thing a visitor sees, since that's what most visitors come to the page to find.
2. `/careers/[id]` (`CareerDetailClient.tsx`) shows one job's full description and requirements, with a link to `/apply?id=<jobId>` .
3. `/apply` (`src/app/apply/page.tsx` , component `ApplyForm`) fetches the specific job (`fetchJob(jobId)`), then renders a 3-step form: candidate details → document upload (`src/components/recruitment/FileUploadZone.tsx`) → review.
4. On submit, `submitApplication(jobId, formData)` tries, in order:
    - `POST /api/recruitment/applications` → proxied to the backend's `POST /api/applications` (protected by the `x-api-key` shared secret, not a login) → backend extracts CV text, uploads files to storage, **runs AI screening synchronously**, writes `Application` + `AIScreeningResult` rows, sends confirmation/notification emails.
    - If that whole path throws, it falls back to the legacy `POST /api/apply` route — a pure Supabase insert with **no AI screening at all**.
5. The candidate gets a confirmation email (`send_application_received_email`) including a link to the self-service status page (`/application-status/[applicationId]` , see 04-hr-portal.md) where they can check their status later without contacting HR.

Where to make changes

CHANGE	FILE
Job board filtering/sorting	`src/app/careers/page.tsx`
Application form fields/steps	`src/app/apply/page.tsx`
What happens when an application is submitted (validation, storage, screening trigger)	`backend/app/routers/applications.py`
CV text extraction (PDF/DOCX parsing)	`backend/app/routers/applications.py` (uses `PyMuPDF` / `python-docx`)
Confirmation/notification email content	`backend/app/services/email_service.py` (`send_application_received_email` , `send_new_application_notification`)

Things worth knowing

- The `x-api-key` protecting `POST /api/applications` is a single shared secret, not per-user — it exists to keep the endpoint from being hit by arbitrary bots, not to authenticate the applicant.
- If the backend is down at submission time, the candidate's application is silently accepted into Supabase instead, and will **not** be AI-screened or visible in the HR portal. See 02-data-model-and-storage.md and 06-security-and-known-gaps.md.
- The confirmation email is sent from a real, repliable address (`recruitment@techspecialistlimited.com`) with a `Reply-To` header pointing at real monitored inboxes, not a "do not reply" address — see 03-configuration-and-integrations.md.

FEATURE — AI CV SCREENING

AI CV Screening

What it does

The moment a candidate submits an application, the backend has GPT-4o read the CV (and cover letter, if provided) against the job's `screening_instructions` and produce a score plus written feedback — before HR ever looks at it.

Walkthrough

1. Triggered inline, synchronously, inside `POST /api/applications` (`backend/app/routers/applications.py`) — not a background job (see 07-deployment-and-operations.md for why that matters operationally).
2. `backend/app/services/ai_screening.py` calls Azure OpenAI GPT-4o with a structured-JSON-response prompt built from the CV text, cover letter text, and the job's requirements/screening instructions.
3. The response is normalized into: `overall_score` , `strengths` , `concerns` , `evidence` , `recommendation` .
4. `backend/app/workers/tasks.py` 's `run_screening()` upserts an `AIScreeningResult` row linked to the application.
5. If the AI call errors for any reason, the system does **not** fail the application — it falls back to a neutral score of 50 with a "needs manual review" note, so a candidate is never silently lost because of an AI outage.

Where to make changes

CHANGE	FILE
Scoring prompt / what the AI weighs	`backend/app/services/ai_screening.py`
Per-job screening emphasis	`job_postings.screening_instructions` , editable from `/hr/jobs/create` and `/hr/jobs/[jobId]` (edit)
Fallback behavior on AI failure	`backend/app/services/ai_screening.py`
Where the score/feedback surfaces to HR	`src/app/hr/applicants/[applicationId]/page.tsx` , `src/app/hr/jobs/[jobId]/applicants/page.tsx`

Things worth knowing

- This step runs **in the request path** of the candidate's submission — the candidate's browser waits for GPT-4o to respond before getting a success confirmation. A slow or degraded Azure OpenAI response directly slows down the application form for the candidate.
- The `raw_response` field is kept on `AIScreeningResult` specifically so a human can audit exactly what the AI said, even if the parsed `strengths` / `concerns` fields don't fully capture it.

FEATURE — AI VOICE INTERVIEW

AI Voice Interview (Stage 2)

What it does

Once HR approves a candidate past the CV screen (or auto-advance does it for them — see below), the candidate gets an emailed magic link to a short AI-conducted voice interview: the AI asks questions on a set of topics, listens to spoken answers, and produces a scorecard.

Two engines exist — the realtime engine is what candidates actually get

	REALTIME ENGINE (PRIMARY)	LEGACY ENGINE (KEPT FOR ROLLBACK)
Mechanism	Azure OpenAI **Realtime API** (`gpt-realtime`) — true voice-to-voice, one persistent bidirectional WebSocket connection held open for the whole interview.	Candidate speaks → Whisper transcribes → GPT-4o decides the next turn → Azure TTS speaks the reply. Turn-based, three sequential API calls per turn.
Backend router	`backend/app/routers/assessment_realtime_ws.py`	`backend/app/routers/assessment.py` (HTTP) and `assessment_ws.py` (WebSocket variant, per-sentence streamed TTS)
Frontend component	`RealtimeAssessmentPortal.tsx`	`LegacyAssessmentPortal.tsx`
Feature flag	`realtime_interview_enabled` — **set to `True` in both local and production config**; this is not an experimental toggle, it's what's actually live	n/a — always reachable directly at `/api/assessment/{token}/ws`, but not what the flag routes candidates to
`conversation_sessions.engine` value	`"realtime"`	`"legacy"`

`src/app/assessment/[token]/page.tsx` (`AssessmentPortalSwitcher`) fetches session metadata from `GET /api/recruitment/assessment/[token]` (which now also returns `interview_max_minutes`) and picks the engine based on what the backend reports. The legacy engine's code is deliberately left in place, untouched, as a fast rollback path if the realtime engine ever needs to be turned off — flip `realtime_interview_enabled` to `False` and restart, no code change needed.

The realtime engine, in detail

Turn detection: semantic VAD, not raw volume

Session config (`backend/app/services/realtime_interviewer.py` 's `build_session_update()`) sets:

- `turn_detection: {"type": "semantic_vad", "eagerness": settings.realtime_vad_eagerness}` — a model-based turn detector that judges whether the candidate has actually *finished their thought*, not just gone quiet. `eagerness: "low"` (the configured default) waits up to ~8 seconds before assuming they're done, so pauses like "let me think... okay, so..." don't get chopped into multiple fake "turns." This replaced a first version that used amplitude-based `server_vad` with a fixed 500ms silence timeout, which was too eager to interrupt candidates mid-thought and too sensitive to quiet non-speech sounds (a throat-clear was enough to trigger `speech_started` and cut the AI off).
- `noise_reduction: {"type": settings.realtime_noise_reduction_type}` — filters ambient audio (room noise, keyboard clicks) *before* it reaches VAD, further reducing false triggers. Default `far_field` (tuned for laptop/room microphones, since headphones aren't guaranteed even though they're recommended in the pre-interview setup screen).

Session instructions are refreshed every turn

The model's system instructions (current topic index, turns-on-topic, time remaining) are re-sent via `conn.session.update(...)` **every turn** — specifically, right before the second `response.create()` call that produces the spoken reply (see "why two model responses per turn" below). This matters because a Realtime API session's instructions are otherwise sticky: sent once at connection time and never refreshed unless you explicitly do it again. An earlier version of this code only sent it once at connect, so the model was working from stale topic/turn state after the very first exchange — a real bug, now fixed.

Why every candidate turn triggers two model responses

The interviewer is instructed to call a function tool (`advance_interview` , in `realtime_interviewer.py`) recording its decision (follow-up / next-topic / end-interview) *before* speaking its reply. The Realtime API's function-calling flow requires this to happen as two separate `response.create()` calls: one response containing only the function call, then (after the backend submits the function's output) a second response containing the actual spoken reply. This is inherent to how tool calls work in the API (mirrors standard chat-completions tool-call semantics) — it's not a bug, but it is real, unavoidable added latency per turn worth knowing about if the interview ever feels laggy.

Graceful time-based wrap-up, not a silent cutoff

Each job has a configurable `interview_max_minutes` (default 20, set on `/hr/jobs/create`). As that soft limit approaches (`minutes_remaining <= 2`), the refreshed session instructions include a `WRAP_UP_NOTICE` telling the model to stop starting new topics, thank the candidate, and call `end_interview` — so the interview ends with an actual spoken closing statement instead of the connection just dying mid-question. A separate, backend-wide **hard safety cap** (`realtime_max_session_seconds`, default 2700s = 45 min) still exists underneath in case a model ignores the nudge; it force-ends the session via `asyncio.wait_for(...)` timing out, with no closing statement, as a last resort.

The completion screen waits for the AI to actually finish talking

The frontend used to flip to the "Interview Ended" screen on a flat 1.5-second guessed delay after the final `audio_done` event — which cut off the AI's closing statement mid-sentence whenever it ran longer than that. It now computes the actual remaining scheduled-audio-buffer duration (`nextStartTimeRef` vs. `AudioContext.currentTime`) and only shows the completed screen once **both** the closing audio has genuinely finished playing **and** the final score has arrived from the backend (`maybeFinishInterview()` in `RealtimeAssessmentPortal.tsx`, gated on `audioFinishedRef` + `finalScoreReceivedRef`). A candidate clicking "End Interview" themselves skips the audio-wait (there's no closing statement to wait for in that path).

Live captions track the audio, not just the final text block

`response.output_audio_transcript.delta` events are forwarded to the frontend as `transcript_delta` messages and appended to the on-screen caption as they arrive, instead of the frontend only receiving one full text block after the entire reply finished generating (which visibly desynced from the audio that had already started playing). The final `transcript` message (sent at `response.done`) still arrives as a resync safety net in case any deltas were dropped.

Candidate-facing countdown timer

The "conversing" screen's top bar shows a live `mm:ss` countdown against the job's `interview_max_minutes`, turning amber under 2 minutes remaining. Purely informational — the actual enforcement is server-side (the wrap-up nudge and hard cap above).

Walkthrough (both engines, conceptually)

1. Candidate opens the emailed link: `/assessment/{token}`. The token is `Application.assessment_token`, a `secrets.token_urlsafe(32)` string — the sole credential for this flow, no login. It can expire (`assessment_expires_at`); an expired link returns HTTP 410 and the candidate sees an "expired" state.
2. A `ConversationSession` row is created (or resumed), seeded with `topics` copied from the job's `stage2_topic_labels` / `stage2_questions`.
3. The interview proceeds topic-by-topic. State machine rules (verified by `backend/tests/test_realtime_advance_topic.py`, the one real unit test in the backend):
    - A "follow-up" response stays on the current topic.
    - "Next topic" advances and resets the turn counter.
    - Hitting `max_turns_per_topic` forces a topic advance even if the model wanted a follow-up.
    - Advancing past the last topic marks the interview done.
    - An explicit "end interview" signal always ends it, regardless of topic position.
4. **Violation detection** (realtime engine only): if the candidate appears to leave the browser tab mid-interview, the session is flagged, the application status becomes `assessment_flagged`, and an audit log entry is written.
5. On completion, `backend/app/services/conversation.py`'s `evaluate_full_conversation()` (shared by both engines) produces a multi-dimension score (communication, technical competency, confidence, problem-solving, relevance, per-topic breakdown, recommendation), written to a new `StageResult` row. This is also what powers the HR dashboard's "Pending Team Decisions" AI-summary widget — see 04-hr-portal.md.
6. The interview auto-finalizes gracefully as `interview_max_minutes` approaches (realtime engine), with `realtime_max_session_seconds` as the hard backstop if the graceful path is ever skipped.

Two ways a candidate reaches stage 2

- **Manual HR approval** (default): HR reviews the CV screening result and clicks approve.
- **Auto-advance** (opt-in per job, configured on `/hr/jobs/create`): if `job_postings.auto_advance_enabled` is true and the candidate's CV screening score meets `auto_advance_pass_mark`, the stage-2 invite is scheduled automatically via a FastAPI `BackgroundTasks` job (`_auto_advance_after_delay` in `backend/app/routers/applications.py`), sent after `auto_advance_delay_minutes` (default 5) — giving HR a window to intervene manually first. The background task re-checks the application is still `pending` before acting, so a manual HR decision made during the delay window wins.

Where to make changes

CHANGE	FILE
Interview state machine (topic advance rules)	`backend/app/services/realtime_interviewer.py` (`advance_topic()`) — has a real unit test, change it with matching test updates
Realtime engine session config, turn detection, wrap-up nudge, system instructions, prompt-injection defenses	`backend/app/services/realtime_interviewer.py`
Realtime WebSocket proxy / event handling / timing enforcement	`backend/app/routers/assessment_realtime_ws.py`
Legacy engine conversation logic / prompts	`backend/app/services/conversation.py`

CHANGE	FILE
Speech-to-text (legacy engine only — realtime does STT natively)	`backend/app/services/audio_evaluation.py` (`transcribe_audio`)
Text-to-speech (legacy engine only — realtime does TTS natively)	`backend/app/services/tts.py`
Per-job interview topics/questions	`job_postings.stage2_topic_labels` / `stage2_questions` , editable from `/hr/jobs/create`
Per-job interview timing	`job_postings.interview_max_minutes` , editable from `/hr/jobs/create`
Auto-advance config	`job_postings.auto_advance_enabled` / `auto_advance_pass_mark` / `auto_advance_delay_minutes` , editable from `/hr/jobs/create`
Global interview timing/tuning	backend config: `topic_time_limit_seconds` , `max_turns_per_topic` , `realtime_max_session_seconds` , `realtime_vad_eagerness` , `realtime_noise_reduction_type`
Live caption sync, completion timing, countdown timer	`src/app/assessment/[token]/RealtimeAssessmentPortal.tsx`

Things worth knowing

- The realtime engine's system instructions explicitly include prompt-injection/jailbreak defenses (`realtime_interviewer.py`) — worth reviewing before changing that prompt, since it's the one place actively defending against a candidate trying to manipulate the interviewer.
- `audio_evaluation.py` 's standalone `evaluate_transcript()` function appears to be legacy/unused — current routers call `conversation.py` 's full-conversation evaluator instead. Confirm before relying on it.
- Interview audio is uploaded to the `assessments` storage container (Azure Blob in prod, local disk in dev) — uploads appear to be best-effort/non-fatal if they fail, so don't assume every session has a recoverable audio file.
- Two full model responses per candidate turn (function-call, then spoken reply) is a known, real latency cost inherent to the current function-calling design — if the interview ever needs to feel snappier than it does today, decoupling the topic-bookkeeping decision from the blocking tool call (e.g. making it a non-blocking background classification instead of a pre-reply gate) is the next lever, not something already done.

FEATURE — HR PORTAL

HR Portal

What it does

The internal back-office where HR staff post jobs, review AI-screened candidates, run the pipeline through to hire/reject, schedule and score final interviews, view analytics, and manage their own team's accounts.

Access

Gated by `src/app/hr/layout.tsx`, which redirects to `/hr/login` unless `isHRAuthenticated()` (a client-side check of `localStorage['hr_token']`). See 06-security-and-known-gaps.md for the important caveat that this is a **UI-level** guard — the real enforcement is the backend's JWT check on each API call.

Pages

ROUTE	PURPOSE
`/hr`	Dashboard — stat cards, active jobs, **Pending Team Decisions** (AI-summarized stage-2 results awaiting a human call — see below)
`/hr/login`, `/hr/reset-password`	Auth (the two pages exempted from the layout guard)
`/hr/jobs`, `/hr/jobs/create`, `/hr/jobs/history`	Create/manage/soft-delete/restore job postings, including AI screening config, interview topics, auto-advance, and interview timing
`/hr/jobs/[jobId]/applicants`	Applicants for one job — **Kanban board** (default) or cards view, bulk actions, archived filter
`/hr/applicants`	All applicants, searchable
`/hr/applicants/[applicationId]`	Full applicant detail: CV screening result, interview transcript/scores, approve/reject/hire, schedule interview, interviewer scorecards, resend assessment link, audit log, delete application
`/hr/interviews`	Interview calendar/list
`/hr/analytics`	Recruitment analytics — overview, pipeline funnel, interview stats, time-to-hire, trends
`/hr/documents`	View/download applicant CVs and cover letters
`/hr/settings`	Branding settings + HR team member management (invite, deactivate, delete)

The pipeline board (`/hr/jobs/[jobId]/applicants`)

A Kanban board (`src/components/recruitment/PipelineBoard.tsx`, native HTML5 drag-and-drop, no external DnD library) with six columns: New, Assessment Sent, Assessment Completed, Interview Scheduled, Hired, Rejected. Dragging a card to a valid destination column opens the matching action (a `ConfirmDialog` for approve/reject/hire, or `ScheduleInterviewModal` for interview scheduling); an invalid drop shows a dismissible banner instead of silently failing. A `viewMode` toggle (default `'board'`) switches to a flat `'cards'` list with checkboxes for bulk selection.

Cards (both board and cards view) show **stalled** and **duplicate** badges, computed client-side in `src/lib/applicant-flags.ts`:

- `isStalled(app)` — no status change in 7+ days, excluding terminal statuses (hired/rejected).
- `isDuplicate(app)` — `application_count_for_email > 1`, a field the backend computes by counting other applications sharing the same candidate email.

A `showArchived` checkbox toggles whether archived applicants are included (`fetchApplicants(jobId, includeArchived)` → `GET /api/hr/applications/{job_id}?include_archived=true`).

Bulk actions

In cards view, selecting multiple applicants surfaces a toolbar for **Archive**, **Unarchive**, or **Reject**, each behind a `ConfirmDialog` (never a native `confirm()`). Calls `POST /api/hr/bulk-action` with `{application_ids: [...], action: "archive"|"unarchive"|"reject"}` — each application gets its own audit log entry, and `reject` also sends the rejection email per candidate.

Interviewer scorecards

`src/components/recruitment/ScorecardPanel.tsx`, shown on the applicant detail page once `applicant.stage >= 2`: interviewer name, four 1–5 sliders (communication, technical, culture fit, problem solving), a recommendation button group (strong yes / yes / no / strong no), and free-text notes. Submits via `POST /api/hr/scorecards`; a list of previously-submitted scorecards for the application loads via `GET /api/hr/scorecards/by-application/{application_id}`. Multiple interviewers can each submit their own scorecard for the same application — see the `interview_scorecards` table in 02-data-model-and-storage.md.

Scheduling a final interview — now with a calendar invite

`ScheduleInterviewModal.tsx` (shared between the board's drag-to-schedule flow and the applicant detail page) calls `createInterview()` → `POST /api/hr/interviews`, which now also builds and attaches a `.ics` calendar file (`backend/app/services/ics_service.py`, base64-encoded via Resend's `attachments` field) so the invitation email drops straight into the candidate's calendar app. The invitation email's "contact us" line and `Reply-To` header point at real monitored addresses (`applicant_reply_to_email`), not the sending address — see 03-configuration-and-integrations.md.

"Pending Team Decisions" (dashboard)

A section on `/hr` (`src/app/hr/page.tsx`) surfaces every application that just finished its stage-2 AI interview and is still sitting in limbo (not hired/rejected, not archived, job not deleted) — pulled from `GET /api/hr/pending-decisions`, which joins the application, its stage-2 `StageResult`, and the job. Each card shows the candidate, score, the AI's structured recommendation (color-coded badge), and a summary, linking straight to the applicant detail page. This is what fixed a real pre-existing bug: `StageResult.ai_feedback` is stored as a raw JSON *string* in the database, but the API was passing it straight through typed as `str | None` while the frontend expected a parsed object — silently rendering nothing for the granular scorecard fields. `_parse_ai_feedback()` in `hr_review.py` now JSON-decodes it before it leaves the backend.

Candidate self-service status page (public, not HR-gated)

`/application-status/[applicationId]` — a public page where a candidate enters the email they applied with and sees a friendly status label (no login, no token). Backed by `GET /api/applications/{application_id}/status?email=...`, which validates the email matches case-insensitively and 404s otherwise (deliberately vague to avoid leaking whether an application ID is valid). A link to this page is included in the "application received" confirmation email.

Single-application delete ("Danger Zone")

The applicant detail page has a destructive "Delete Application" action, behind a `ConfirmDialog`, calling `DELETE /api/hr/applications/{application_id}`. Unlike the bulk archive/reject actions, this is a hard, cascading delete — it removes the `AIScreeningResult`, all `StageResult` rows, the `ConversationSession`, all `Interview` rows, and the `AuditLog` history for that application, then the `Application` itself. There is no undo.

Auto-advance configuration (job creation)

On `/hr/jobs/create`, a "Stage 2 Invitation Flow" section lets HR pick Manual Review (default — HR clicks approve) or Auto-Advance (candidates who clear a configured CV-screening pass mark get the stage-2 invite automatically after a configurable delay, no click needed). See 03-ai-voice-interview.md for how the delay/override window works, and the same page's "Maximum Interview Duration" field for per-job interview timing.

Walkthrough — reviewing and advancing a candidate

1. HR opens `/hr/applicants/[applicationId]`, which calls `fetchApplicantDetail()` → `GET /api/recruitment/hr/applicants/[applicationId]` → backend `GET /api/hr/applications/detail/{application_id}`.
2. The page renders the AI screening score/strengths/concerns, and, if a stage-2 interview has happened, the transcript and scorecard (`TranscriptViewer`, `ScoreCircle`, `ScoreBar` in `src/components/recruitment/`).
3. HR clicks approve, reject, or **hire** → `reviewApplication()` / `approveApplication()` → `POST /api/recruitment/hr/review|approve/[applicationId]` → backend `POST /api/hr/review/{id}` or `/api/hr/approve/{id}`. Approving past stage 1 generates a new `assessment_token`, sets `assessment_sent_at` / `assessment_expires_at`, and emails the candidate the stage-2 interview link. Rejecting sends a rejection email. Hiring sends a congratulations email and sets `status = "hired"`. All three write an `AuditLog` entry.
4. HR can resend the assessment link (`resendAssessment()` → `POST /api/recruitment/hr/resend/[applicationId]`) if the candidate's link expired.
5. After a successful stage-2 interview, HR schedules a final human interview (`createInterview()` → `POST /api/recruitment/hr/interviews`), which emails an invitation (with calendar invite attached) and sets `applications.status = 'interview_scheduled'`. One or more interviewers can then submit a scorecard.
6. HR marks the application hired or rejected, or archives it if it's gone stale.

Team management (`/hr/settings`)

- Inviting a new HR user creates an `HrUser` row with an unusable placeholder password and emails a set-password (reset-token) link.
- Deactivating or deleting the **last remaining active HR account is blocked** by the backend, to prevent HR from locking itself out entirely.

Where to make changes

CHANGE	FILE
Dashboard stats / pending decisions	`backend/app/routers/hr_review.py` (`GET /api/hr/stats`, `GET /api/hr/pending-decisions`)

CHANGE	FILE
Applicant list/detail data shape, bulk action, delete	`backend/app/routers/hr_review.py`
Pipeline board UI, drag-and-drop rules	`src/components/recruitment/PipelineBoard.tsx`
Stalled/duplicate detection logic	`src/lib/applicant-flags.ts`
Scorecard form / list	`src/components/recruitment/ScorecardPanel.tsx` , `backend/app/routers/scorecards.py`
Calendar invite generation	`backend/app/services/ics_service.py`
Approve/reject/hire/interview email content	`backend/app/services/email_service.py`
Analytics calculations	`backend/app/routers/analytics.py`
Branding fields available in Settings	`backend/app/services/settings_service.py` (`OVERRIDABLE_KEYS`)
Team member invite/deactivate/delete rules	`backend/app/routers/hr_settings.py`
Candidate status page	`src/app/application-status/[applicationId]/page.tsx` , `backend/app/routers/applications.py` (`GET /{id}/status`)

Things worth knowing

- Every mutating HR action (approve, reject, hire, schedule interview, bulk action, delete, scorecard submission, clear applications) writes to `audit_logs` — `/hr/applicants/[applicationId]` surfaces this per-applicant via `fetchAuditLogs()` . Deleting an application deletes its audit history too (the cascading delete above); bulk-clearing a job's applications does not preserve audit history either.
- `POST /api/recruitment/hr/clear/[jobId]` bulk-deletes all applications (and cascading screening/stage/conversation/interview rows) for a job. This is destructive and irreversible — the UI confirms before calling it using the app's custom `ConfirmDialog` component, not the browser's native `confirm()` . The single-application delete above is the same pattern, scoped to one applicant.
- No system modals (`confirm()` / `alert()` / `prompt()`) are used anywhere in the HR portal — every destructive or blocking action goes through `ConfirmDialog` or a purpose-built modal component. If you find a native browser dialog anywhere in this codebase, it's a bug, not a design choice.

FEATURE — AI READINESS ASSESSMENT

AI Readiness Assessment (Lead Magnet)

What it does

A public, self-serve quiz at `/ai-readiness-assessment` that scores a visitor's organization on "AI readiness" across several pillars, shows them results, and offers to email them a full PDF report — capturing their email as a marketing lead in the process. This feature is **entirely independent of the recruitment platform** — different AI provider, different data, no shared tables.

Walkthrough

1. `src/app/ai-readiness-assessment/page.tsx` runs a client-side phase state machine: `landing → questions → results`, persisting progress to `localStorage['ts-ai-assessment']` so a visitor can resume if they navigate away.
2. `AssessmentLanding.tsx` — pillar selection. `AssessmentQuestions.tsx` (with `AssessmentStepper.tsx` / `AssessmentProgress.tsx`) — question flow, driven by question data in `src/data/assessment`. `AssessmentResults.tsx` — computes and displays the score.
3. On reaching results, `sendAssessmentLeadEmail()` (`src/lib/emailjs.ts`) fires a client-side EmailJS notification, and the results are posted to `POST /api/assessment` → proxied to backend `POST /api/ai-readiness/submit`, which stores an `AiReadinessAssessment` row.
4. If the visitor requests the full report, `POST /api/assessment/send-report` runs:
    - `generateAIReport()` (`src/lib/ai-report.ts`) — calls Anthropic Claude (`claude-sonnet-4-6`, strict JSON-schema output) to write the report content; falls back to Groq (`llama-3.3-70b-versatile`) if Anthropic isn't configured; returns `null` if neither is.
    - `ReportDocument` (`src/lib/pdf-report.tsx`, built with `@react-pdf/renderer`) renders that content into a formatted PDF (executive summary, priority gaps, pillar analysis, action plan, success metrics, branded footer).
    - The PDF is emailed via Resend from `reports@techspecialistlimited.com`.
    - This work is deferred with Next.js's `after()` so the visitor's HTTP request returns immediately and the slow AI+PDF+email work happens after the response.

Where to make changes

CHANGE	FILE
Quiz questions/pillars	`src/data/assessment`
Scoring logic	`AssessmentResults.tsx`
Report content/prompt	`src/lib/ai-report.ts` (`buildPrompt`)
PDF layout/branding	`src/lib/pdf-report.tsx`
Lead storage	`backend/app/routers/ai_readiness.py`

Things worth knowing

- This is the only feature in the codebase that calls Anthropic or Groq — everything else AI-related (screening, interviews) goes through Azure OpenAI.
- If neither `ANTHROPIC_API_KEY` nor `GROQ_API_KEY` is configured, report generation silently returns `null` — confirm the request-flow handles that gracefully (i.e. doesn't send a broken email) before assuming this path always works.

FEATURE — ADMIN LEADS DASHBOARD

Admin Leads Dashboard

What it does

`/admin/assessments` lists everyone who has completed the AI Readiness Assessment quiz, so marketing/sales can follow up. Supports filtering by readiness level, CSV export, marking a lead as "followed up," and deleting a lead.

Walkthrough

1. On mount, `src/app/admin/assessments/page.tsx` calls `GET /api/admin/assessments` (`src/app/api/admin/assessments/route.ts`), which proxies to backend `GET /api/ai-readiness/results`.
2. The table shows each lead's email, company, score, level (`AI Explorer` / `AI Builder` / `AI Accelerator` / `AI Leader`), and follow-up status.
3. **Filter by level** — client-side.
4. **CSV export** — built client-side from the already-fetched data (a `Blob`), no separate export endpoint.
5. **Toggle followed-up** — `PATCH /api/admin/assessments` → backend `PATCH /api/ai-readiness/results/{id}`.
6. **Delete** — confirmed via the app's `ConfirmDialog` component, then `DELETE /api/admin/assessments` → backend `DELETE /api/ai-readiness/results/{id}`.

Where to make changes

CHANGE	FILE
Table columns / filters	`src/app/admin/assessments/page.tsx`
CSV export format	same file
Lead list/update/delete logic	`backend/app/routers/ai_readiness.py`

This is the most important thing to know about this feature

There is no authentication or authorization anywhere in this path. No `layout.tsx` or guard component wraps `/admin`, `src/app/api/admin/assessments/route.ts` performs no auth check before proxying, and the backend's `/api/ai-readiness/*` router requires no auth either. Anyone who discovers the URL can view, export, or delete every captured lead.

A `ADMIN_SECRET_TOKEN` environment variable exists in `.env.local`, suggesting access control was planned, but it is not referenced anywhere in the codebase — it does nothing today.

If this dashboard holds real, current lead data, treat closing this gap as a priority — see 06-security-and-known-gaps.md for the fix options.

API REFERENCE

API Reference

How to read this

The frontend (`src/app/api/**`) and backend (`backend/app/routers/**`) each expose their own routes. Almost every frontend recruitment route is a thin proxy to a backend route via `proxyToBackend()` (`src/app/api/recruitment/proxy.ts`), forwarding the `Authorization` header and any body/query params as-is. Where a frontend route also has a Supabase fallback, it's noted — those fallbacks are the ones without a backend-equivalent auth check (see 06-security-and-known-gaps.md).

Auth column values: **none** (public), **api-key** (`x-api-key` shared secret), **HR JWT** (backend `verify_hr_token`), **token** (candidate magic-link token).

Frontend routes (`src/app/api/**`)

ROUTE	BACKEND TARGET	FALLBACK	AUTH (AS ENFORCED)
`POST /api/apply`	— (legacy, Supabase-only)	n/a — this *is* the fallback	none
`GET/PATCH/DELETE /api/admin/assessments`	`/api/ai-readiness/results[/{id}]`	none	**none**
`POST /api/assessment`	`/api/ai-readiness/submit`	none	none
`POST /api/assessment/send-report`	— (calls Anthropic/Groq + Resend directly)	none	none
`GET/POST /api/recruitment/jobs` , `GET /api/recruitment/jobs/[jobId]`	`/api/jobs[/{id}]`	Supabase `jobs` (GET only)	none
`POST /api/recruitment/applications`	`/api/applications`	Supabase `applications` insert + storage upload	api-key (backend only; **fallback has none**)
`GET /api/recruitment/applications/[applicationId]/status`	`/api/applications/{id}/status`	none	none (email query param must match)
`GET/POST /api/recruitment/assessment/[token]`	`/api/assessment/{token}[/{action}]`	none	token
`POST /api/recruitment/hr/auth`	`/api/auth/login`	hardcoded `HR_EMAIL` / `HR_PASSWORD` check	none (this route *issues* the JWT)
`POST /api/recruitment/hr/auth/change-password`	`/api/auth/change-password`	none	HR JWT
`POST /api/recruitment/hr/auth/forgot-password`	`/api/auth/forgot-password`	none	none (by design — must work pre-login)
`POST /api/recruitment/hr/auth/reset-password`	`/api/auth/reset-password`	none	reset token
`GET/POST /api/recruitment/hr/jobs` , `GET/PUT /api/recruitment/hr/jobs/[jobId]`	`/api/jobs` (POST/PUT)	Supabase `jobs` read/write	HR JWT (backend only; **fallback has none**)
`PUT /api/recruitment/hr/jobs/[jobId]/soft-delete` , `/restore`	`/api/jobs/{id}/soft-delete` , `/restore`	none	HR JWT
`GET /api/recruitment/hr/jobs/history`	`/api/jobs/history`	none	HR JWT
`GET /api/recruitment/hr/applications/[jobId]`	`/api/hr/applications/{job_id}`	none	HR JWT
`DELETE /api/recruitment/hr/applications/[jobId]` (route param reused as an application id, not a job id — same file handles both)	`/api/hr/applications/{application_id}`	none	HR JWT
`GET /api/recruitment/hr/applicants/[applicationId]`	`/api/hr/applications/detail/{id}`	none	HR JWT
`GET /api/recruitment/hr/applicants/[applicationId]/document`	`/api/hr/applications/{id}/document`	none	HR JWT

ROUTE	BACKEND TARGET	FALLBACK	AUTH (AS ENFORCED)
`POST /api/recruitment/hr/review/[applicationId]`	`/api/hr/review/{id}` (action: `approve` | `reject` | `hire`)	none	HR JWT
`POST /api/recruitment/hr/approve/[applicationId]`	`/api/hr/approve/{id}`	none	HR JWT
`POST /api/recruitment/hr/resend/[applicationId]`	`/api/hr/resend/{id}`	none	HR JWT
`POST /api/recruitment/hr/bulk-action`	`/api/hr/bulk-action` (action: `archive` | `unarchive` | `reject`)	none	HR JWT
`POST /api/recruitment/hr/clear/[jobId]`	`/api/hr/clear/{job_id}`	none	HR JWT
`GET /api/recruitment/hr/pending-decisions`	`/api/hr/pending-decisions`	none	HR JWT
`POST /api/recruitment/hr/scorecards`	`/api/hr/scorecards`	none	HR JWT
`GET /api/recruitment/hr/scorecards/by-application/[applicationId]`	`/api/hr/scorecards/by-application/{id}`	none	HR JWT
`GET /api/recruitment/hr/audit-logs/[applicationId]`	`/api/hr/audit-logs/{id}`	none	HR JWT
`GET /api/recruitment/hr/stats`	`/api/hr/stats`	none	HR JWT
`GET /api/recruitment/hr/analytics/{overview,interviews,pipeline,time-to-hire,trends}`	`/api/hr/analytics/*`	none	HR JWT
`GET/POST /api/recruitment/hr/interviews`, `GET/PUT /api/recruitment/hr/interviews/[interviewId]`	`/api/hr/interviews[/{id}]`	none	HR JWT
`GET /api/recruitment/hr/interviews/{all,upcoming}`	`/api/hr/interviews/{all,upcoming}`	none	HR JWT
`GET /api/recruitment/hr/interviews/by-application/[applicationId]`	`/api/hr/interviews/by-application/{id}`	none	HR JWT
`GET/PUT/POST /api/recruitment/hr/settings`	`/api/hr/settings`	none	HR JWT
`GET/POST /api/recruitment/hr/users`, `PATCH/DELETE /api/recruitment/hr/users/[userId]`	`/api/hr/users[/{id}]`	none	HR JWT

Backend routes (`backend/app/routers/**`)

`jobs.py` — prefix `/api/jobs`

- `POST /api/jobs` — create a job posting, including AI screening config, interview topics, auto-advance settings, and `interview_max_minutes` (HR JWT)
- `GET /api/jobs` — list jobs (public; `show_all` / `deleted` query flags)
- `GET /api/jobs/history` — soft-deleted jobs (public)
- `GET /api/jobs/{job_id}` — one job + applicant count (public)
- `PUT /api/jobs/{job_id}` — update status/department/location/type/is_closed/auto_advance_*/interview_max_minutes (HR JWT)
- `PUT /api/jobs/{job_id}/soft-delete` / `/restore` (HR JWT)

`applications.py` — prefix `/api/applications`

- `POST /api/applications` — candidate submits (multipart: job_id, name, email, cv, optional cover letter; requires `x-api-key`). If the job has auto-advance enabled and the CV screening score clears the pass mark, schedules a delayed stage-2 invite via `BackgroundTasks`.
- `GET /api/applications/{application_id}` — fetch one (public)
- `GET /api/applications/{application_id}/status?email=...` — candidate self-service status check (public; email must match case-insensitively or 404s)

`hr_review.py` — prefix `/api/hr`

- `GET /api/hr/stats` — dashboard counts (HR JWT)
- `GET /api/hr/pending-decisions` — stage-2-complete applications awaiting a hire/reject/interview decision, with parsed AI recommendation/summary (HR JWT)
- `GET /api/hr/applications/{job_id}` — applicants for a job, sorted by score; `include_archived` query flag; includes per-email duplicate-application counts (HR JWT)
- `GET /api/hr/applications/detail/{application_id}` — full applicant detail (HR JWT)
- `POST /api/hr/approve/{application_id}` — advance stage, send assessment invite (HR JWT)
- `POST /api/hr/review/{application_id}` — `approve`, `reject`, or `hire` (HR JWT)
- `POST /api/hr/bulk-action` — `archive` / `unarchive` / `reject` across multiple `application_ids` in one call, each individually audit-logged (HR JWT)

- `DELETE /api/hr/applications/{application_id}` — hard, cascading delete (screening result, stage results, conversation session, interviews, audit log, then the application itself) — irreversible (HR JWT)
- `POST /api/hr/resend/{application_id}` — regenerate/resend assessment link (HR JWT)
- `POST /api/hr/clear/{job_id}` — bulk-delete a job's applications and related rows (HR JWT)
- `GET /api/hr/applications/{application_id}/document` — stream CV/cover letter (HR JWT)
- `GET /api/hr/audit-logs/{application_id}` — audit trail (HR JWT)

`scorecards.py` — prefix `/api/hr`

- `POST /api/hr/scorecards` — submit a structured interviewer scorecard (validates `recommendation` against a fixed enum: `strong_yes` / `yes` / `no` / `strong_no`; audit-logs the submission) (HR JWT)
- `GET /api/hr/scorecards/by-application/{application_id}` — list scorecards for an application, newest first (HR JWT)

`analytics.py` — prefix `/api/hr/analytics` (all HR JWT)

- `GET /overview`, `/interviews`, `/pipeline`, `/time-to-hire`, `/trends`

`interviews.py` — prefix `/api/hr/interviews` (stage-3 human interviews, all HR JWT)

- `POST /` — schedule, sends invitation email, logs audit
- `GET /by-application/{application_id}`, `/upcoming`, `/all`
- `PUT /{interview_id}` — update, logs audit

`assessment.py` — prefix `/api/assessment` (session metadata + legacy HTTP engine, token auth)

- `GET /{token}` — session metadata: candidate/job info, topics, `engine` (`"realtime"` / `"legacy"`, drives which frontend portal loads), `interview_max_minutes` (410 if link expired)
- `POST /{token}/start` — begin/resume, returns streamed TTS audio (legacy engine)
- `POST /{token}/respond` — candidate audio in, transcribe → next AI turn → streamed TTS reply (legacy engine)
- `POST /{token}/end` — force-end and score (legacy engine)

`assessment_ws.py` — prefix `/api/assessment` (legacy engine, WebSocket)

- `WS /{token}/ws` — bidirectional streaming audio, per-sentence TTS

`assessment_realtime_ws.py` — prefix `/api/assessment` (Realtime engine — the primary interview experience, see 04-features/03-ai-voice-interview.md)

- `WS /{token}/realtime-ws` — one persistent voice-to-voice session against Azure OpenAI's Realtime API (`gpt-realtime`) for the whole interview, gated by `realtime_interview_enabled` (currently `True` in production). Enforces per-job `interview_max_minutes` (soft, graceful wrap-up) and `realtime_max_session_seconds` (hard backstop).

`auth.py` — prefix `/api/auth` (HR login)

- `POST /login` — returns JWT (HS256, 8h)
- `POST /forgot-password` — always returns a generic success message (avoids user enumeration)
- `POST /reset-password` — consumes reset token
- `POST /change-password` (HR JWT)

`hr_settings.py` — prefix `/api/hr`

- `GET/PUT /settings` — branding/notification config (HR JWT)
- `GET/POST /users` — list / invite HR accounts (HR JWT)
- `PATCH/DELETE /users/{user_id}` — activate/deactivate/delete, blocks removing the last active account (HR JWT)

`ai_readiness.py` — prefix `/api/ai-readiness` (no auth on any of these)

- `POST /submit` — public quiz submission
- `GET /results` — list all submissions
- `PATCH /results/{assessment_id}` — mark followed up
- `DELETE /results/{assessment_id}`

Misc

- `GET /health` — liveness check, returns `{"status": "ok"}`

SECURITY & KNOWN GAPS

Security, Auth, and Known Gaps

Summary for non-technical readers

There are two separate login systems in this application (one for HR staff, effectively none for the AI-readiness leads admin panel), and neither uses Supabase or Firebase's built-in authentication — both are custom-built. The HR system is reasonably solid when the Python backend is reachable, but has a **weaker fallback path** that activates during backend outages. The leads admin panel currently has **no access control at all**. This document lists what's verified true in the code today, so decisions about what to fix (and in what order) can be made with facts rather than assumptions.

How HR authentication actually works

1. **Real path (backend reachable):** `POST /api/recruitment/hr/auth` proxies to backend `POST /api/auth/login`, which checks the submitted password against a bcrypt hash in the `hr_users` table and, on success, issues a genuine JWT (HS256, signed with the backend's `jwt_secret`, 8-hour expiry). Every subsequent HR-scoped backend endpoint validates this JWT via `verify_hr_token()` (`backend/app/auth.py`).
2. **Fallback path (backend unreachable — HTTP 502):** `src/app/api/recruitment/hr/auth/route.ts` checks the submitted credentials against the frontend's own `HR_EMAIL` / `HR_PASSWORD` env vars (defaults `hr@company.com` / `admin123` if unset — **check these are actually set to non-default values in production**), and mints a token that **looks like a JWT but isn't**: `signature = base64url(JWT_SECRET + header + payload)`, i.e. string concatenation encoded, not HMAC-SHA256. This token would not pass a real JWT signature check, but nothing in the frontend re-validates it either — the frontend simply trusts it exists.
3. `src/lib/auth.ts` stores whichever token was issued in `localStorage['hr_token']` and attaches it as `Authorization: Bearer <token>` to every subsequent HR API call (`recruitment-api.ts`'s `authHeaders()`).
4. **The `/hr/*` page guard (`src/app/hr/layout.tsx`) only checks that *some* token exists in `localStorage`** — it does not verify it's valid. It is a UI-rendering guard, not a security boundary. The actual security boundary is the backend's `verify_hr_token()` on each API call.

The gap this creates

When a mutating HR route has a Supabase fallback (currently: `POST/PUT /api/recruitment/hr/jobs*`), that fallback code path performs **no auth check whatsoever** before writing to Supabase — it doesn't even look at the `Authorization` header. In practice, this means: if the backend is down, anyone who can reach `/api/recruitment/hr/jobs` (not just logged-in HR staff) can create or modify a job posting. This is a narrower gap than it sounds, because it only matters during a specific failure mode (backend down), but it's real and worth fixing — either by having those fallback routes also check for a valid token, or by removing the write fallback and simply failing the request when the backend is unreachable.

The AI Readiness leads admin panel has no auth at all

`/admin/assessments`, `src/app/api/admin/assessments/route.ts`, and backend `backend/app/routers/ai_readiness.py`'s `/api/ai-readiness/*` endpoints have **zero authentication or authorization** at any layer. Anyone who discovers the URL can:

- View every captured lead (email, company name, score)
- Export all of it as CSV
- Mark leads as followed up
- **Delete leads permanently**

An `ADMIN_SECRET_TOKEN` env var exists in `.env.local`, which strongly suggests this was meant to be protected and the wiring was never finished. This is the single highest-priority finding in this documentation set if the leads data is real and current.

Straightforward fix options (not yet implemented — for whoever picks this up):

- Reuse the existing HR JWT mechanism: require the same `Authorization: Bearer` header and `verify_hr_token()` check on the backend's `/api/ai-readiness/*` routes, and add the same `layout.tsx`-style client guard to `/admin`.
- Or, simplest: check `ADMIN_SECRET_TOKEN` as a shared secret (like the recruitment `x-api-key` pattern) if a full login isn't wanted for this one panel.

Other verified auth mechanisms (working as designed)

- **Public application submission** (`POST /api/applications`): a static shared secret via `x-api-key`, checked against backend `settings.api_key`. This isn't candidate authentication — it's a bot/abuse gate on a public form, and that's an appropriate use for a shared secret.
- **Candidate voice interview** (`/api/assessment/{token}/...`): a single-use-context magic-link token (`secrets.token_urlsafe(32)`), no password. Appropriate for a one-shot, emailed, time-limited flow — just be aware anyone who has the email link has the interview access, so link expiry (`assessment_expires_at`) is the only real time boundary.
- **Password reset** (`POST /api/auth/forgot-password`): always returns a generic success message regardless of whether the email exists, correctly avoiding user enumeration.

Data-integrity gap: the Supabase fallback and AI screening

Covered in depth in 02-data-model-and-storage.md, but worth restating here because it's a data-integrity issue with a security-adjacent cause (an unauthenticated-by-design fallback path): an application submitted while the backend is down lands in Supabase, **skips AI screening**, and is invisible to the HR portal until manually reconciled. This isn't a confidentiality/access problem, but it is a silent-data-loss-shaped problem worth the same level of attention.

What is *not* a gap (confirmed by code, not assumption)

- No Supabase Auth or Firebase Auth is used anywhere for HR or admin — this is intentional custom auth, not a misconfigured integration.
- The Supabase client only ever uses the anon key; no service-role key is exposed anywhere in `src/`.
- Password reset tokens are hashed (SHA-256) before storage (`hr_auth_service.py`) with a 1-hour TTL — not stored in plaintext.
- HR passwords are bcrypt-hashed, not plaintext, in the `hr_users` table.

Recommended priority order

1. **Add auth to `/admin/assessments` and its API routes** — currently fully open, highest exposure for the least effort to fix.
2. **Close or auth-gate the Supabase fallback write paths for HR jobs** — narrower exposure (only during backend outages) but touches production job data.
3. **Confirm `HR_EMAIL` / `HR_PASSWORD` are not left at their code-level defaults** (`hr@company.com` / `admin123`) in whatever environment serves production traffic.
4. **Decide whether the fallback JWT-like token minting is acceptable long-term**, or whether the frontend should simply fail login when the backend is unreachable rather than issuing a weaker credential.

DEPLOYMENT & OPERATIONS

Deployment and Operations

Summary

The frontend has a clear CI/CD path to Azure Web App. The backend does not — it has a `Dockerfile` but no CI workflow in this repository; it's deployed **manually**, by a human running two Azure CLI commands (documented below). There's also declared-but-unused infrastructure (Celery/Redis) worth knowing about before assuming background job processing exists.

Running locally

Frontend:

```
npm install
npm run dev0      # next dev — note: not the default "dev" script name
```

Requires `.env.local` with, at minimum, `RECRUITMENT_API_URL=http://localhost:8000` and the Supabase keys for the fallback paths to work.

Backend:

```
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Requires `backend/.env` (see `backend/.env.example` for the full list). With no `DATABASE_URL` override, it defaults to a local SQLite file (`backend/recruitment.db`) and `dev_mode`, which also switches file storage to local disk and email sending to console-logging instead of calling Resend.

On every startup (`app/main.py`'s lifespan hook), the backend:

1. Creates any missing tables (`Base.metadata.create_all`).
2. Runs `run_migrations()` — hand-rolled `ALTER TABLE` statements for columns added after initial deployment (see below).
3. Runs `seed_defaults()` — creates a default HR account (`hr@company.com`, password from `HR_PASSWORD`) and default `app_settings` rows if they don't exist.

Deployment

Frontend: `.github/workflows/main_techspecialist-limited.yml` builds and deploys to an **Azure Web App** named `Techspecialist-Limited` on every push to `main` (via `azure/webapps-deploy`). Build-time secrets injected: `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `NEXT_PUBLIC_ASSESSMENT_WS_URL`.

A `.vercel/repo.json` project link also exists in the repo, which implies a Vercel project may also be (or have been) connected. **Confirm with whoever manages hosting which one is the live, authoritative deployment** before making assumptions in an incident — don't assume Azure is the only place this is running.

Backend: has a `Dockerfile` (`python:3.12-slim`, installs `requirements.txt`, runs `uvicorn app.main:app --host 0.0.0.0 --port 8000`), and no matching GitHub Actions workflow exists in `.github/workflows/` — deployment is entirely manual. Confirmed live setup:

- **Where it runs:** Azure Web App for Containers, name `techspecialist-api`, resource group `techspecialist`, region `canadacentral`. URL: `https://techspecialist-api-aub0eafrb0d4hcbq.canadacentral-01.azurewebsites.net`.
- **Image registry:** Azure Container Registry `techspecialistacr`, image `recruitment-api:latest`.
- **Deploy sequence** (run from the repo root, needs the Azure CLI logged in with access to the `techspecialist` resource group):

  ```
  az acr build --registry techspecialistacr --image recruitment-api:latest ./backend
  az webapp restart --name techspecialist-api --resource-group techspecialist
  ```

  `az acr build` uploads and builds from the **local** `backend/` **directory on disk** — not from git — so uncommitted local changes get deployed too if the working tree isn't clean when you run it. Check `git status backend/` first if that matters to you.
- **Two operational quirks worth knowing before you trust a "successful" deploy:**
  1. On Windows, `az acr build`'s local log streaming can crash mid-build with `UnicodeEncodeError: 'charmap' codec can't encode characters` (a `colorama` / `cp1252` console issue) — this is cosmetic, the remote build usually keeps running fine. Don't treat a crashed local stream as a failed build; check the real status with `az acr task list-runs --registry techspecialistacr --top 1 --query "[0].status" -o tsv` instead.
  2. **A single** `az webapp restart` **doesn't reliably pull the new image.** `/health` can return `200 OK` while the container is still silently serving the *previous* cached image (the App Service host doesn't always force a fresh `docker pull` on a plain restart when the tag name — `latest` — hasn't changed). After restarting, verify a field or behavior that only exists in the new code (e.g. `curl .../api/jobs` and check for a newly-added response field) before declaring the deploy done — don't just trust the health check. If the new code isn't there yet, restart again.

- **Database migrations** run automatically on backend startup (see below), so a fresh Postgres column added in this deploy takes effect on the same restart — no separate migration step needed.

Database migrations — read before touching the schema

Despite a `backend/alembic/` directory existing, **it is completely empty** — no `alembic.ini`, no `env.py`, no versioned migration files, and no Alembic import anywhere in the code. This project does **not** use Alembic in practice.

The actual mechanism is `backend/app/migrate.py`, run automatically on every app startup:

- New tables are created automatically from the SQLAlchemy models.
- New *columns* on existing tables are added via dialect-aware `ALTER TABLE` statements hardcoded in `run_migrations()` (a separate SQLite branch and Postgres branch). As of this writing, these patch in: `job_postings.is_deleted/is_closed/department/location/type/auto_advance_enabled/auto_advance_pass_mark/auto_advance_delay_minutes/interview_max_minutes`, `applications.is_archived/assessment_sent_at/assessment_expires_at`, `conversation_sessions.engine`. The `interview_scorecards` table itself is new but created wholesale from the ORM model (`Base.metadata.create_all`), not via a hand-rolled `ALTER TABLE` — only *added columns on existing tables* need the manual migration step.

Practical implication: if you need to add a new column to an existing table, you add it to the model *and* add a matching `ALTER TABLE ... ADD COLUMN` line to `run_migrations()` — there's no `alembic revision --autogenerate` workflow to lean on. If you need to drop a column, rename a column, or do anything more complex than "add a nullable column," this hand-rolled approach won't help you and you'll need to write the migration by hand (and probably introduce real Alembic at that point).

Background jobs: declared but not implemented

`requirements.txt` lists `celery[redis]` and `redis`, and `config.py` defines `redis_url`. **No Celery app, task decorator, beat schedule, or worker process exists anywhere in the codebase.** The one function that looks like a background task — `backend/app/workers/tasks.py`'s `run_screening()` — is called directly and synchronously (`await run_screening(...)`) inline inside `POST /api/applications`, blocking that HTTP request until the GPT-4o call completes.

Practical implications:

- Don't assume there's a job queue you can push work onto — there isn't one yet, despite the dependency being present.
- CV screening latency is directly the candidate's page-load latency for the application form. If Azure OpenAI is slow, the application form is slow.
- If a real queue is ever introduced (there's a clear future use case here — moving AI screening off the request path), it would need actual Celery/RQ/similar wiring added, not just relying on the currently-listed dependencies.

Testing

There is no `pytest`/CI-wired test suite for the backend. What exists:

- `backend/test_api.py`, `test_full_flow.py`, `test_submit.py` — standalone scripts that exercise a *running* server (job creation → application submission → screening → approval), useful as manual smoke tests or as a reference for the request/response shapes, but they don't run in CI.
- `backend/tests/test_realtime_advance_topic.py` — the one genuine unit test (no pytest framework, run via `python tests/test_realtime_advance_topic.py`), covering the interview topic/turn-count state machine (`advance_topic()` in `realtime_interviewer.py`). If you change interview-flow logic, run this and consider extending it.

No frontend test suite was found either (`npm run test` in the CI workflow is a no-op — `--if-present` and no `test` script exists in `package.json`).

Known operational debt, summarized

ITEM	RISK IF IGNORED
No Alembic, hand-rolled column migrations	Schema changes beyond "add a nullable column" require manual work and are easy to get wrong across SQLite/Postgres
Celery/Redis declared but unused	AI screening runs in-request; a slow AI provider directly slows down the public application form
No backend CI/CD workflow found	Production backend deployment process is undocumented — confirm and record it
No automated test suite (backend or frontend)	Regressions in the interview state machine or screening logic won't be caught before production
Two Supabase/backend data stores for jobs & applications	See 02-data-model-and-storage.md — silent data loss during backend outages